

ASCLEPIOS O.T

Système d'Orientation Clinique Multi-Agents

Rapport de Projet

Travaux Pratiques — LangGraph · FastAPI · MCP · React

Par : Omar Tarrouzi

Encadrant : Pr. Mohamed YOUSSEF

Date de rendu : 19 juin 2026

Table des matières

1	Introduction et contexte	2
1.1	Objectif du projet	2
1.2	Problématique adressée	2
2	Technologies utilisées	3
2.1	Vue d’ensemble de la pile technique	3
2.2	Justification des choix techniques	4
2.2.1	LangGraph vs approches alternatives	4
2.2.2	MCP — Model Context Protocol	4
2.2.3	FastAPI comme couche d’intégration	4
3	Architecture du projet	4
3.1	Structure générale du système	4
3.2	Arborescence du projet	4
3.3	Schéma d’architecture globale	5
3.4	Structure des dossiers — captures d’écran	5
4	Graphe LangGraph	5
4.1	Description du graphe	5
4.2	Transitions conditionnelles	7
4.3	Visualisation du graphe — LangGraph Studio	7
4.4	Mécanisme Human-in-the-Loop	7
5	API FastAPI — Endpoints	9
6	Démonstration pas à pas — Interface ASCLEPIOS O.T	9
6.1	Écran 1 — Saisie des symptômes initiaux	10
6.2	Écran 2 — Interview IA (5 questions)	11
6.3	Écran 3 — Revue du médecin traitant (Human-in-the-Loop)	12
6.4	Écran 4 — Rapport final	12
7	État d’avancement	13
7.1	Progression par phase	14
7.2	Progression globale	15
7.3	Difficultés rencontrées et solutions apportées	15
8	Conclusion et perspectives	15
	Annexe — Ressources et références	17

1. Introduction et contexte

Avertissement éthique et académique

Ce système est exclusivement un exercice académique. Il ne constitue pas un dispositif médical agréé et ne doit jamais fournir de diagnostic définitif. Les termes appropriés sont : *orientation clinique préliminaire*, *synthèse clinique*, *recommandation intermédiaire*. Tout rapport généré mentionne obligatoirement : « *Ce système ne remplace pas une consultation médicale.* »

1.1. Objectif du projet

ASCLEPIOS O.T (Orientation Thérapeutique) est un système multi-agents médical simulant un workflow d'orientation clinique complet. Il s'inscrit dans le cadre du cours de systèmes distribués intelligents et met en œuvre les technologies modernes d'orchestration d'agents IA. j'ai voulu ajouté ma touche personnelle en lui donnant une esthétique mythologique.

Le système réalise les fonctions suivantes :

- Recueil des symptômes d'un patient via un agent conversationnel interactif
- Production d'une synthèse clinique préliminaire structurée
- Validation humaine par un médecin traitant (*Human-in-the-Loop*)
- Génération d'un rapport final structuré et traçable
- Exposition de l'ensemble via une API REST FastAPI
- Interface utilisateur moderne construite avec React

1.2. Problématique adressée

La conception d'un système de ce type soulève plusieurs défis techniques :

- **Coordination d'agents** : comment orchestrer plusieurs agents spécialisés (Supervisor, DiagnosticAgent, PhysicianReview, ReportAgent) de manière fiable ?
- **Interruptions contrôlées** : comment suspendre et reprendre l'exécution d'un graphe d'agents pour intégrer des entrées humaines sans perte d'état ?
- **Interopérabilité** : comment exposer un graphe d'agents complexe via une API REST consommable par n'importe quel frontend ?
- **Extensibilité** : comment brancher des outils médicaux externes (bases de données médicaments, détecteurs de red flags) via un protocole standardisé ?

2. Technologies utilisées

2.1. Vue d'ensemble de la pile technique

TABLE 1 – Technologies et leurs rôles dans le projet

Technologie	Rôle dans le projet	Niveau d'usage
Python 3.11+	Langage principal du backend. Gestion du graphe, des agents, de l'API et des outils.	Principal
LangGraph	Orchestration du graphe d'agents multi-étapes. Gestion des transitions conditionnelles, du checkpointing et des interruptions (<i>Human-in-the-Loop</i>).	Central
LangChain	Abstraction LLM, définition des tools et des prompts. Interface unifiée vers OpenAI/Anthropic.	Élevé
FastAPI	Exposition de l'API REST. Gestion des sessions, des endpoints de consultation et de la revue médecin. Documentation Swagger auto-générée.	Élevé
MCP (Model Context Protocol)	Protocole standard ouvert (Anthropic) pour connecter des outils médicaux externes : base médicaments, vérificateur d'interactions, détecteur de red flags.	Moyen
React (TypeScript)	Interface utilisateur moderne et réactive. Gestion des 4 écrans du workflow clinique avec animations Framer Motion.	Élevé
LangGraph Studio	Débogage visuel du graphe d'agents. Inspection des états, timeline des transitions, panneau d'interruption.	Moyen
LangSmith	Traçabilité des appels LLM. Monitoring des coûts et de la latence en développement.	Optionnel

2.2. Justification des choix techniques

2.2.1. LangGraph vs approches alternatives

LangGraph a été retenu pour plusieurs raisons déterminantes :

- **État persistant** : le `MedicalState` (`TypedDict`) circule entre tous les agents et est sérialisé à chaque nœud via un *checkpoint* en mémoire, permettant la reprise exacte après interruption.
- **Interruptions natives** : la fonction `interrupt()` suspend l’exécution du graphe à n’importe quel point — essentiel pour les dialogues patient et la revue médecin — sans nécessiter de files de messages asynchrones.
- **Transitions conditionnelles** : le routage dynamique (`add_conditional_edges`) permet au Supervisor de décider de la prochaine étape en fonction de l’état courant.

2.2.2. MCP — Model Context Protocol

MCP standardise l’intégration d’outils externes dans les agents LLM. Dans ce projet, le serveur MCP expose : la base de données de médicaments, le vérificateur d’interactions médicamenteuses, et le détecteur de red flags cliniques. Cette architecture rend les outils indépendants du graphe et réutilisables dans d’autres contextes.

2.2.3. FastAPI comme couche d’intégration

FastAPI joue le rôle de passerelle entre le graphe LangGraph (Python asynchrone) et le frontend React. Chaque endpoint correspond à une phase du workflow : démarrage, réponse patient, revue médecin, récupération du rapport.

3. Architecture du projet

3.1. Structure générale du système

L’architecture suit une séparation claire en trois couches :

- **Couche Orchestration** : le graphe LangGraph avec ses quatre nœuds agents
- **Couche API** : FastAPI expose le graphe via des endpoints REST stateless
- **Couche Présentation** : l’interface React consomme l’API et guide l’utilisateur

3.2. Arborescence du projet

La structure adoptée sépare strictement le backend (`backend/app/`), le serveur MCP (`mcp_server/`), le frontend (`frontend/`) et les tests (`tests/`). Cette organisation est



FIGURE 1 – vue d'ensemble

requis par LangGraph Studio qui attend l'objet `graph` à l'emplacement déclaré dans `langgraph.json`.

3.3. Schéma d'architecture globale

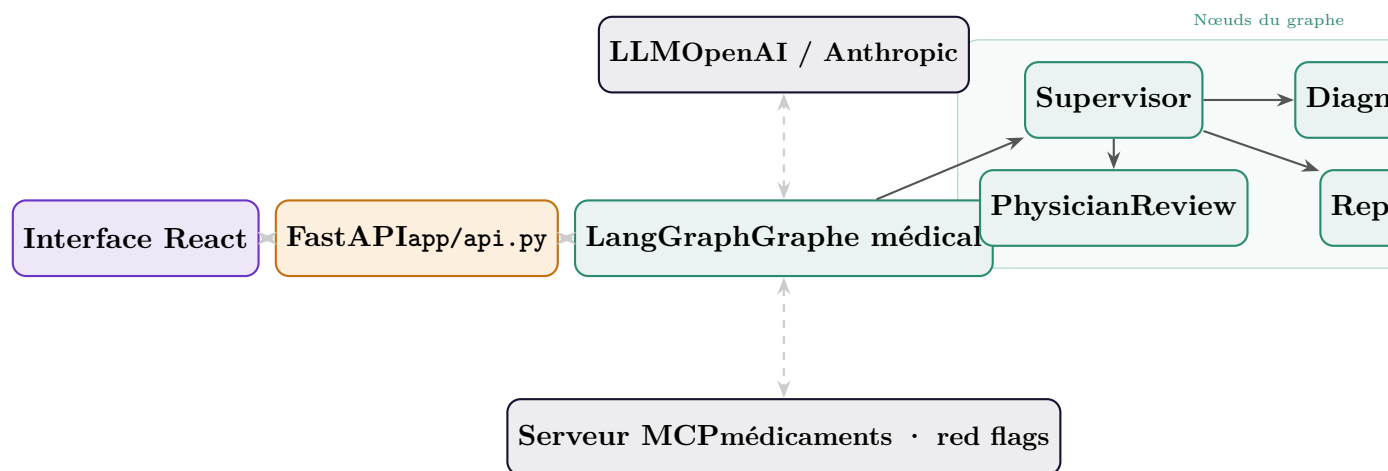


FIGURE 2 – Architecture générale du système ASCLEPIOS O.T

3.4. Structure des dossiers — captures d'écran

4. Graphe LangGraph

4.1. Description du graphe

Le graphe médical est construit avec un StateGraph dont l'état partagé (`MedicalState`) circule entre quatre nœuds principaux :

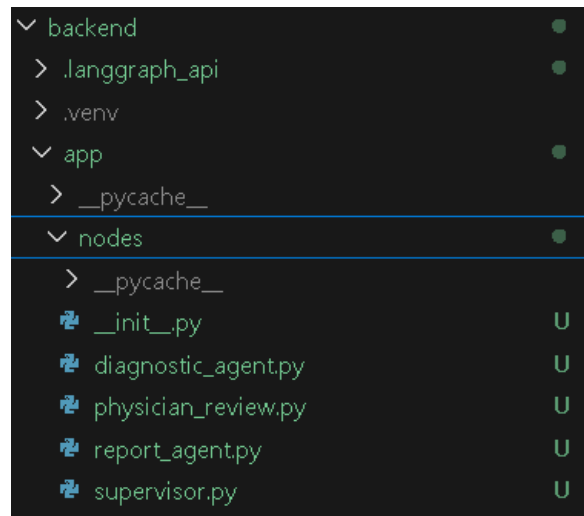


FIGURE 3 – Backend nodes

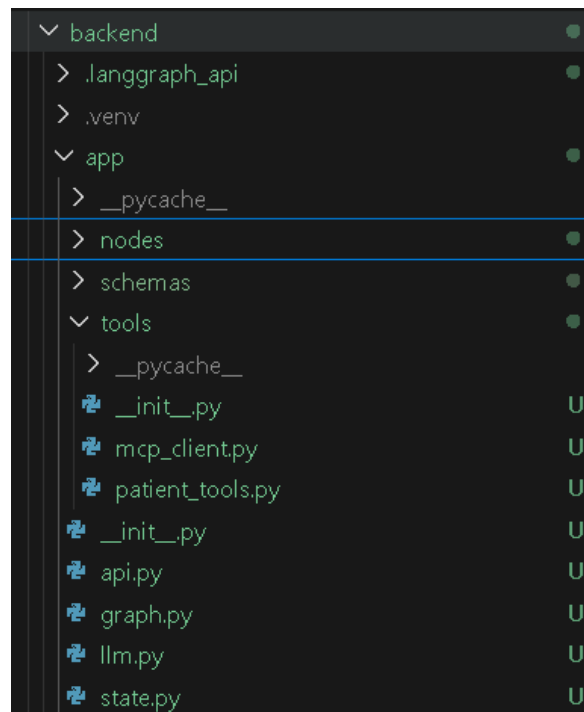


FIGURE 4 – backend tools

TABLE 2 – Nœuds du graphe LangGraph

Nœud	Rôle
supervisor	Chef d’orchestre. Lit l’état et route vers le nœud approprié via un LLM + logique déterministe de fallback.
diagnostic_agent	Pose 5 questions successives au patient via <code>interrupt()</code> , analyse les réponses et produit la synthèse clinique.
physician_review	Nœud Human-in-the-Loop. Suspend l’exécution en attente de l’avis du médecin traitant.
report_agent	Génère le rapport final structuré à partir de l’ensemble des informations collectées.
tools	ToolNode LangGraph. Exécute les tools <code>ask_patient</code> et <code>recommend_interim_care</code> .

4.2. Transitions conditionnelles

Logique de routage du Supervisor

Le Supervisor inspecte quatre champs de l’état pour décider :

1. `question_count < 5` → **diagnostic_agent** (continuer les questions)
2. `question_count = 5` et pas de synthèse → **diagnostic_agent** (produire synthèse)
3. Synthèse produite et pas de traitement médecin → **physician_review**
4. Traitement médecin reçu et pas de rapport → **report_agent**
5. Rapport produit → **FINISH**

4.3. Visualisation du graphe — LangGraph Studio

4.4. Mécanisme Human-in-the-Loop

Le système exploite deux niveaux d’interruption :

- **Niveau tool** — `interrupt()` dans le tool `ask_patient` : suspension dynamique avec un payload contenant la question, l’index et les instructions. Reprise via `Command(resume=answer)`.
- **Niveau nœud** — `interrupt_before=["physician_review"]` : interruption déclarative avant l’entrée dans le nœud de revue médecin. L’état est sérialisé et

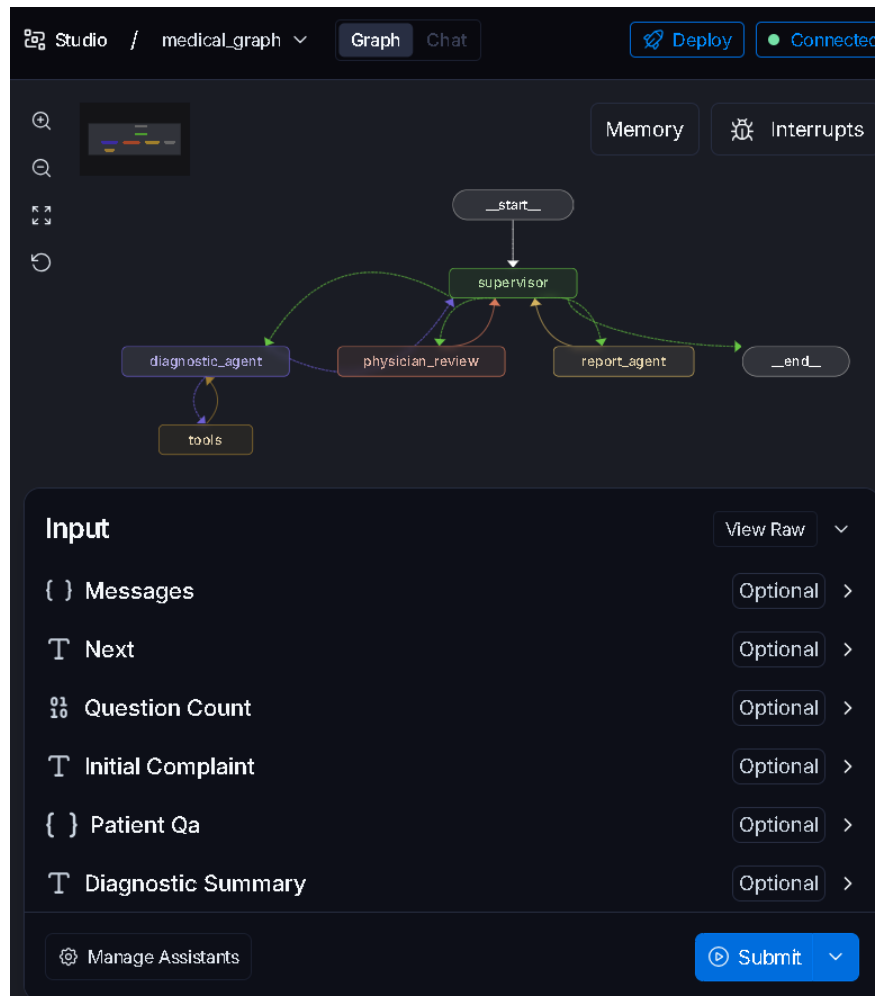


FIGURE 5 – graph langgraph

récupéré par l’API FastAPI pour être présenté au médecin.

5. API FastAPI — Endpoints

TABLE 3 – Endpoints de l’API REST

Méthode	Endpoint	Description
POST	/consultation/start	Lance le workflow. Retourne <code>thread_id</code> et la première question patient.
POST	/consultation/resume	Soumet la réponse du patient. Le graphe reprend et retourne la question suivante ou le statut <code>awaiting_physician</code> .
POST	/physician/review	Soumet l’avis du médecin (traitement, approbation, commentaires). Déclenche la génération du rapport.
GET	/consultation/{id}	Retourne l’état complet d’une consultation (statut, questions, synthèse, rapport).
GET	/consultation/{id}/report	Retourne le rapport final structuré.
GET	/health	Vérification de l’état de l’API.

6. Démonstration pas à pas — Interface ASCLEPIOS O.T

L’interface React **ASCLEPIOS O.T** guide l’utilisateur à travers quatre écrans successifs, matérialisés dans la barre de progression temporelle en haut de la page.

Palette de l’interface

L’interface adopte une esthétique *Alchemical Void* : fond indigo profond, accent ambre/feu pour les actions, jade pour les données cliniques, violet pour les éléments d’architecture occulte. Arrière-plan animé avec un champ d’étoiles tri-couches et des nébuleuses respirantes.

6.1. Écran 1 — Saisie des symptômes initiaux

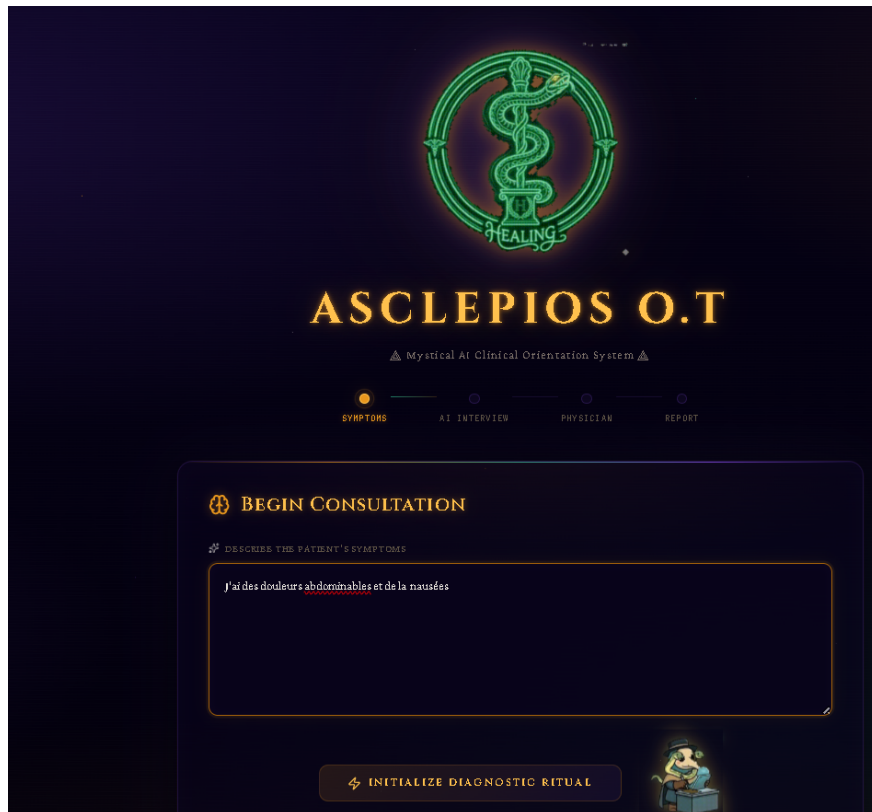


FIGURE 6 – Écran 1 : saisie du motif de consultation

Éléments visibles :

- Logo animé ASCLEPIOS O.T (futur : phoenix avec fond supprimé)
- Indicateur de progression : nœud *Symptoms* actif
- Zone de texte mystique avec placeholder clinique
- Bouton avec effet shimmer ambre et icône *Zap*

6.2. Écran 2 — Interview IA (5 questions)



FIGURE 7 – L'agent DiagnosticAgent pose 5 questions séquentielles standardisées. Chaque réponse est envoyée via POST /consultation/resume. L'historique des échanges s'accumule dans une interface de chat. Le compteur $Qn/5$ s'incrémente à chaque échange.

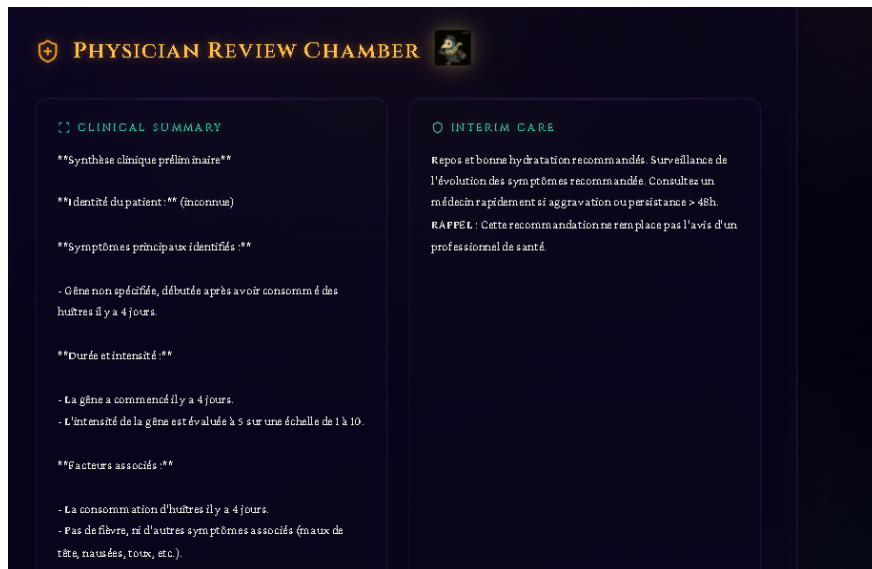


FIGURE 8 – Questions

Questions posées par l'agent :

1. Durée des symptômes (heures, jours, semaines)
2. Intensité de la gêne (échelle 1–10)
3. Présence de fièvre et température relevée
4. Symptômes associés (céphalées, nausées, toux...)
5. Médicaments en cours et allergies connues

6.3. Écran 3 — Revue du médecin traitant (Human-in-the-Loop)

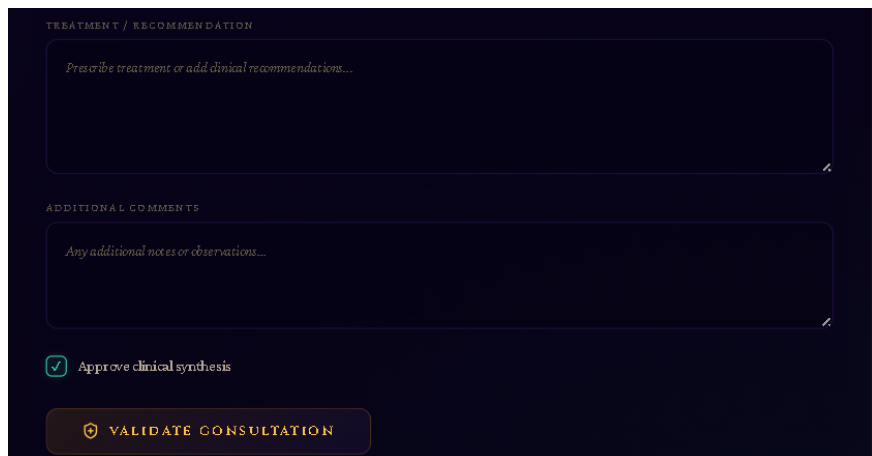


FIGURE 9 – revue du médecin traitant

Éléments affichés :

- Carte *Clinical Summary* : synthèse produite par le DiagnosticAgent
- Carte *Interim Care* : recommandation de soins intermédiaire (avec détection automatique des red flags cliniques)
- Champ libre *Treatment / Recommendation*
- Champ *Additional Comments*
- Checkbox *Approve clinical synthesis*
- Bouton *Validate Consultation*

6.4. Écran 4 — Rapport final

Le ReportAgent génère un rapport structuré intégrant : l'en-tête, le motif, la synthèse, la recommandation intermédiaire, l'avis médecin et la conclusion. La mention « *Ce système ne remplace pas une consultation médicale.* » est insérée obligatoirement.

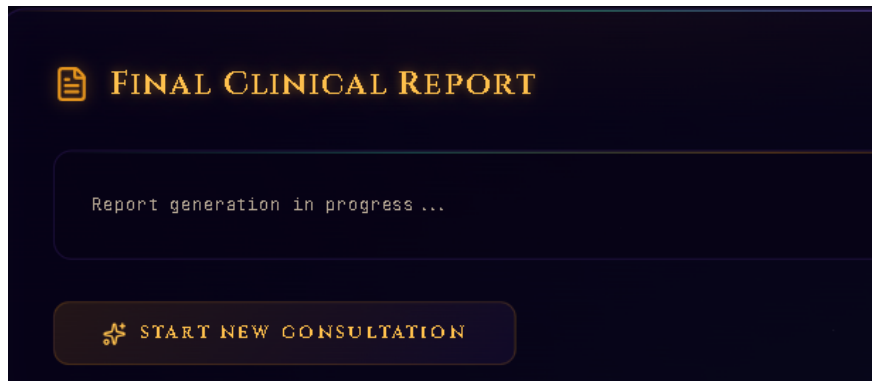


FIGURE 10 – rapport

Structure du rapport généré :

1. En-tête (date, référence de consultation)
2. Motif de consultation
3. Synthèse clinique préliminaire
4. Recommandation intermédiaire
5. Avis du médecin traitant
6. Conclusion et recommandations finales
7. Avertissement légal obligatoire

7. État d'avancement

7.1. Progression par phase

TABLE 4 – Avancement des phases du TP

#	Phase	Composants	Statut
1	Environnement	Structure dossiers, venv, requirements, .env, langgraph.json	Terminé
2	Architecture & état	state.py (MedicalState), schemas/models.py (Pydantic)	Terminé
3	Graphe de base	Supervisor, DiagnosticAgent, ReportAgent, patient_tools	Terminé
4	Human-in-the-Loop	PhysicianReview, interrupt(), Command(resume=...)	Terminé
5	Assemblage graphe	graph.py, transitions conditionnelles, checkpointer	Terminé
6	Intégration MCP	Serveur MCP, get_drug_info, check_drug_interaction, get_red_flags	En cours
7	FastAPI	Tous les endpoints, CORS, gestion des erreurs	Terminé
8	Frontend React	4 écrans, animations, design system Alchemical Void	En cours
9	Tests	Cas 1 respiratoire, Cas 2 red flags, Lang-Graph Studio	À faire
10	Livraison	README, bonus PDF export, bonus Docker Compose	À faire

7.2. Progression globale

100Phase 1 — Environnement

100Phase 2 — Architecture & état

100Phase 3 — Graphe de base

100Phase 4 — Human-in-the-Loop

100Phase 5 — Assemblage graphe

60 Phase 6 — Intégration MCP

100Phase 7 — API FastAPI

75 Phase 8 — Frontend React

10 Phase 9 — Tests

5 Phase 10 — Livraison

7.3. Difficultés rencontrées et solutions apportées

- **Gestion de l'état entre interruptions** : le champ `question_count` n'était pas incrémenté correctement lors de la reprise depuis `interrupt()`. Résolu en s'appuyant sur le `MemorySaver` checkpointer et la lecture de l'état via `graph.get_state(config)`.
- **Synchronisation API-Graphe** : le frontend ne savait pas si le graphe était suspendu sur une question patient ou sur la revue médecin. Résolu en inspectant `state.tasks[0].interrupts[0].value["type"]`.
- **Design system cohérent** : la palette initiale vert néon + or créait un conflit de luminosité. Résolu par l'adoption d'une hiérarchie tri-chromatique : ambre (actions), jade (données), violet (profondeur).

8. Conclusion et perspectives

Le projet **ASCLEPIOS O.T** démontre la faisabilité d'un système d'orientation clinique multi-agents orchestré avec LangGraph. Les mécanismes d'interruption (`interrupt()`)

et `interrupt_before`) permettent une intégration naturelle du facteur humain dans un graphe d'agents automatisé, sans compromis sur la traçabilité de l'état.

Perspectives de développement :

- **Bonus export PDF** : génération du rapport final en PDF via ReportLab, téléchargeable depuis l'interface React.
- **Bonus Docker Compose** : conteneurisation complète des trois services (backend, MCP server, frontend) pour un déploiement reproductible.
- **Phoenix animé** : remplacement du logo texte par une image phoenix générée avec Gemini, animée avec PixVerse et fond supprimé, pour renforcer l'identité visuelle mystique du projet.
- **Enrichissement MCP** : connexion à une base de données médicaments réelle (OpenFDA, RxNorm) via le serveur MCP.
- **Tests d'intégration** : exécution des deux scénarios définis (syndrome respiratoire et red flags) via pytest avec assertions sur le rapport final.

Rappel éthique

Ce système demeure un exercice académique. Tout déploiement réel nécessiterait une certification médicale, une validation clinique rigoureuse et le respect des réglementations sur les dispositifs médicaux (MDR, FDA 21 CFR Part 11).

Annexe — Ressources et références

- Documentation LangGraph : <https://langchain-ai.github.io/langgraph/>
- LangGraph Studio : <https://studio.langchain.com>
- MCP (Model Context Protocol) : <https://modelcontextprotocol.io>
- FastAPI : <https://fastapi.tiangolo.com>
- LangSmith (débogage) : <https://smith.langchain.com>
- Framer Motion (animations React) : <https://www.framer.com/motion/>